Omni-Nexos: Android Release Notes (Google Play)

Android only. Development is done on a Mac with the iOS simulator, so these notes assume you do **not** use Android Studio. Android builds are made with EAS, and testing is done on a real Android phone.

1. Working from an iOS-first setup

- **You do not need Android Studio or the Android SDK.** Build Android in the EAS cloud: `eas build --platform android --profile preview`.
- Only use `--local` for Android if you install the Android SDK and a Java JDK. The cloud build is simpler and has the same automatic build numbers.
- `npx expo run:android` also needs the Android SDK. Skip it. Test Android with an EAS preview build instead.
- Android is a different platform from iOS, so test it on a real Android phone before release, especially forms, camera, location, notifications and the tab bar.

2. Build numbers (automatic)

- Android calls it **versionCode**. EAS raises it on every production build (autoIncrement is on in eas.json).
- Do not edit versionCode in app.json. Check the next number with `eas build:version:get`.
- The user-facing **version** (1.0.0) is changed by you only for a real release.

3. Before the first release

1. **Package name:** replace the placeholder com.anonymous.reactnativeomninexos with your real one (example: com.omniscience.omninexos). It cannot change after the first upload.
2. **Permissions:** add camera, location, photos and notifications plugins with accurate texts in app.json, only for what the app really uses. Google also asks about these in the Data safety form.
3. **Privacy policy URL:** required by Google Play.
4. **Account deletion:** if users can create accounts, Google expects an in-app way to delete the account and a web link for it.
5. **Icon and assets:** the adaptive icon is already set in app.json. The Play listing also needs a 512 x 512 icon, a feature graphic (1024 x 500) and phone screenshots.
6. Run `npx expo-doctor` and test a preview build on a real phone.

```
sed -i '' 's/com.anonymous.reactnativeomninexos/com.omniscience.omninexos/g' app.json
```

4. Test build on your Android phone

```
eas build --platform android --profile preview
```

- The preview profile makes an **.apk**. When the build finishes, EAS gives a link or QR code. Open it on the phone and install.
- On first install the phone asks you to allow installing apps from this source. Allow it for the browser you used.
- On the first Android build, EAS creates a signing keystore for you. Accept it and let EAS manage credentials.

5. Publishing to Google Play

1. Create a Google Play Console developer account (one-time fee, about 25 USD) and verify your identity.
2. Create the app in Play Console. Complete the store listing, content rating, target audience, Data safety form and privacy policy.
3. Build the store bundle: `eas build --platform android --profile production`. This makes an **.aab** signed with the EAS keystore.
4. Upload the **first** .aab manually in Play Console to an internal testing track. Google does not allow the very first upload through the API.
5. For later uploads, automate with `eas submit --platform android --profile production`. It needs a Google service account JSON key. The command walks you through creating it.
6. New personal developer accounts may have to run a closed test with a minimum number of testers for a set period before production access. These rules change, so check the current requirements in Play Console help.
7. When testing passes, promote the release to production and submit it for review.

6. Keep your signing key safe

- Losing the upload key makes updates hard. EAS stores the keystore for you. Download a backup with `eas credentials`, choose Android, and save the file somewhere safe.
- Google Play App Signing is on by default for new apps, which adds a second layer of protection.

7. Releasing fixes later

- **JavaScript-only change:** `eas update --channel production --message "what changed"`. Reaches Android users without a Play review, on builds with the same version.
- **Native change** (new library, plugin or permission): bump **version** in app.json, run a new production build, and submit it.

8. Commands

```
eas build --platform android --profile preview      # test .apk
eas build --platform android --profile production   # store .aab
eas submit --platform android --profile production  # upload to Play
eas build:version:get                               # next build number
eas credentials                                     # backup keystore
eas update --channel production --message "fix"     # OTA update
```